



Záródolgozat

Készítették:

- Szoboszlai Csongor 13/B
- Tilaji Dániel 13/B
- Tóth Kornél 13/B

Konzulens:

Horváth István

**Miskolc
2026.**

Miskolci SZC Kandó Kálmán Informatikai Technikum
Miskolci Szakképzési Centrum
Szoftverfejlesztő és tesztelő szak

Teszt- Dokumentáció

Készítették:

- Szoboszlai Csongor 13/B
- Tilaji Dániel 13/B
- Tóth Kornél 13/B

2025-2026

Tartalomjegyzék

Tartalomjegyzék

Bevezetés	1
Cél.....	1
Tesztelési környezet.....	1
Backend tesztelése	2
Frontend tesztelése	3
Teszteredmények Összegzése	5
Backend teszteredmény:	5
Frontend teszt futás közben:	5
Mellékletek.....	6

NutriTrack TesztDokumentáció

Bevezetés

A **NutriTrack** projekt fejlesztése során fontos szerepet kapott a rendszer működésének **ellenőrzése** és a **hibák kiszűrése**. A tesztelés célja annak biztosítása volt, hogy az alkalmazás egyes funkciói a tervezett módon működjenek, és a felhasználók számára megbízható működést biztosítsanak.

A projekt során a tesztelés **két** fő területre oszlott. Egyrészt **backend** oldalon **unit tesztek** készültek a **kritikus** funkciók ellenőrzésére, másrészt **frontend** oldalon **Selenium** segítségével történt felületi tesztelés.

Cél

A tesztelés célja az volt, hogy a rendszer **legfontosabb funkciói ellenőrzött környezetben vizsgálhatók legyenek**. Ennek során külön figyelmet kaptak azok a **kritikus** funkciók, amelyek közvetlenül befolyásolják a **rendszer biztonságát és használhatóságát**.

A **backend** oldalon elsősorban az olyan **kritikus** folyamatok kerültek tesztelésre, mint a **bejelentkezés**.

Tesztelési környezet

A **NutriTrack** projekt fejlesztése során fontos szerepet kapott a rendszer működésének **ellenőrzése** és a **hibák kiszűrése**. A tesztelés célja annak biztosítása volt, hogy az alkalmazás egyes funkciói a tervezett módon működjenek, és a felhasználók számára megbízható működést biztosítsanak.

A projekt során a tesztelés **két** fő területre oszlott. Egyrészt **backend** oldalon **unit tesztek** készültek a **kritikus** funkciók ellenőrzésére, másrészt **frontend** oldalon **Selenium** segítségével történt felületi tesztelés.

NutriTrack TesztDokumentáció

Backend tesztelése

A backend tesztek célja az volt, hogy a rendszer szerveroldali funkciói külön-külön ellenőrizhetők legyenek. Az unit tesztek segítségével megvizsgálható, hogy az egyes metódusok a megfelelő bemenetek esetén a várt eredményt adják-e vissza.

A tesztelés során elsősorban a felhasználókezeléshez és hitelesítéshez kapcsolódó funkciók kerültek ellenőrzésre, mivel ezek a rendszer működése szempontjából kiemelten fontos elemek.

```
13 namespace NutriTrack.Controllers.Tests
14 {
15     [TestClass]
16     public class RegistryControllerTests
17     {
18         private TesztContext _context;
19         private RegistryController _controller;
20         private IConfiguration _config;
21         private JwtSettings _jwtSettings;
22
23         [TestInitialize]
24         public void Init() ...
25
26         [TestMethod]
27         public async Task Login_ValidUser_ReturnsOk() ...
28
29         [TestMethod]
30         public async Task Login_UnverifiedEmail_ReturnsUnauthorized() ...
31
32         [TestMethod]
33         public async Task Register_ValidUser_CreatesUserWithToken() ...
34
35         [TestMethod]
36         public async Task Register_DuplicateUsername_ReturnsBadRequest() ...
37
38         [TestMethod]
39         public async Task Register_DuplicateEmail_ReturnsBadRequest() ...
40     }
41 }
```

NutriTrack TesztDokumentáció

Frontend tesztelése

A frontend tesztelés során Selenium került alkalmazásra, amely lehetővé teszi a felhasználói felületen végzett műveletek automatizált ellenőrzését. A Selenium segítségével szimulálható a felhasználó viselkedése, például az űrlapmezők kitöltése, a gombokra kattintás, valamint az oldalakon történő navigáció.

A módszer előnye, hogy nemcsak az egyes elemek jelenlétét vizsgálja, hanem a teljes felhasználói folyamat működését is képes ellenőrizni.

```
def test_login_success():
    # **1. Chrome beállítások optimalizálva**
    options = Options()
    options.add_argument("--start-maximized")
    options.add_argument("--disable-blink-features=AutomationControlled")
    driver = webdriver.Chrome(options=options)

    wait = WebDriverWait(driver, 10)

    try:
        print("1. Navigálás a bejelentkezési oldalra...")
        driver.get("http://localhost:5173/login")

        print("2. Login adatok megadása...")
        username_input = wait.until(EC.presence_of_element_located((By.NAME, "username")))
        password_input = driver.find_element(By.NAME, "password")
        submit_button = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")

        username_input.clear()
        password_input.clear()
        username_input.send_keys("Ading")
        password_input.send_keys("Asdasd1")

        print("3. Bejelentkezés...")
        submit_button.click()

        print("4. Várakozás MainPage-re...")
        wait.until(EC.url_contains("/MainPage"))
        print("✅ Sikeres bejelentkezés!")

        driver.save_screenshot("login_success.png")

        print("5. Profil setup...")
        profile_button = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".step-button.profile")))
        profile_button.click()
```

```

print("6. Profil modal kitöltése...")
modal = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".modal-content")))

height_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Height')]/following-sibling::input]")
weight_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Current')]/following-sibling::input]")
target_weight_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Target')]/following-sibling::input]")

height_input.clear()
height_input.send_keys("175")
weight_input.clear()
weight_input.send_keys("70")
target_weight_input.clear()
target_weight_input.send_keys("65")

driver.save_screenshot("profile_complete.png")
save_button = modal.find_element(By.XPATH, "//*[button[@type='submit']]")
driver.execute_script("arguments[0].click();", save_button)

print("✅ TELJES TESZT SIKERES!")
driver.save_screenshot("final_success.png")

except Exception as e:
    print(f"❌ HIBA: {e}")
    print(f"URL: {driver.current_url}")
    print(f"Title: {driver.title}")
    driver.save_screenshot("error_screenshot.png")
    raise

finally:
    time.sleep(3)
    driver.quit()

if __name__ == "__main__":
    test_login_success()

```

NutriTrack TesztDokumentáció

Teszteredmények Összegzése

A végrehajtott backend és frontend tesztek alapján megállapítható, hogy a rendszer legfontosabb funkciói megfelelően működnek. A backend unit tesztek sikeresen ellenőrizték a kritikus szerveroldali logikát, míg a Selenium teszt a felhasználói felület működéséről adott visszajelzést.

A tesztelés hozzájárult a rendszer megbízhatóságának növeléséhez, valamint segített az esetleges hibák korai felismerésében. A vizsgálatok alapján a NutriTrack alkalmazás alapvető funkciói megfelelően használhatók.

Backend teszteredmény:

Test run finished: 5 Tests (5 Passed, 0 Failed, 0 Skipped) run in 3,2 sec			
Test	Duration	Traits	Er
▲ ✓ NutriTrackTests1 (5)	3,1 sec		
▲ ✓ NutriTrack.Controllers.Tests (5)	3,1 sec		
▲ ✓ RegistryControllerTests (5)	3,1 sec		
✓ Login_UnverifiedEmail_Returns...	1,6 sec		
✓ Login_ValidUser_ReturnsOk	626 ms		
✓ Register_DuplicateEmail_Return...	285 ms		
✓ Register_DuplicateUsername_R...	265 ms		
✓ Register_ValidUser_CreatesUser...	281 ms		

Frontend teszt futás közben:

The screenshot displays the NutriTrack web application interface. A modal window titled "Basic Data Entry" is centered on the screen, allowing users to input their personal data. The modal contains three input fields: "Height (cm)" with the value "175", "Current Weight (kg)" with the value "70", and "Target Weight (kg)" with the value "65". Below these fields are two buttons: a green "SAVE" button and a grey "Cancel" button. The background of the application shows a dark green header with the "NutriTrack" logo and navigation links: "Home", "Diary", "Food Search", "Calorie Calculator", "Statistics", and a red "Logout" button. On the left side, there are buttons for "Set Up Profile" and "Calorie". On the right side, there is a "Daily Tip" section with the text "Exercise daily!" and a "Help" section with expandable questions like "How does calorie calculation work?", "Why do I need to enter my target weight?", and "Can I change my data later?".

NutriTrack TesztDokumentáció

Mellékletek

Frontend teszt selenium kód:

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import Select
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.chrome.options import Options

def test_login_success():
    # **1. Chrome beállítások optimalizálva**
    options = Options()
    options.add_argument("--start-maximized")
    options.add_argument("--disable-blink-features=AutomationControlled")
    driver = webdriver.Chrome(options=options)

    wait = WebDriverWait(driver, 10)

    try:
        print("1. Navigálás a bejelentkezési oldalra...")
        driver.get("http://localhost:5173/login")

        print("2. Login adatok megadása...")
        username_input = wait.until(EC.presence_of_element_located((By.NAME, "username")))
        password_input = driver.find_element(By.NAME, "password")
        submit_button = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")

        username_input.clear()
        password_input.clear()
        username_input.send_keys("Ading")
        password_input.send_keys("Asdasd1")

        print("3. Bejelentkezés...")
        submit_button.click()

        print("4. Várakozás MainPage-re...")
        wait.until(EC.url_contains("/MainPage"))
```

```

print("✅ Sikeres bejelentkezés!")

driver.save_screenshot("login_success.png")

print("5. Profil setup...")
profile_button = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".step-button.profile")))
profile_button.click()

print("6. Profil modal kitöltése...")
modal = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".modal-content")))

height_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Height')]/following-sibling::input]")
weight_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Current')]/following-sibling::input]")
target_weight_input = modal.find_element(By.XPATH, "//*[label[contains(text(),'Target')]/following-sibling::input]")

height_input.clear()
height_input.send_keys("175")
weight_input.clear()
weight_input.send_keys("70")
target_weight_input.clear()
target_weight_input.send_keys("65")

driver.save_screenshot("profile_complete.png")
save_button = modal.find_element(By.XPATH, "//*[button[@type='submit']]")
driver.execute_script("arguments[0].click();", save_button)

print("✅ TELJES TESZT SIKERES!")
driver.save_screenshot("final_success.png")

```

```

except Exception as e:
    print(f"❌ HIBA: {e}")
    print(f"URL: {driver.current_url}")
    print(f"Title: {driver.title}")
    driver.save_screenshot("error_screenshot.png")
    raise

finally:
    time.sleep(3)
    driver.quit()

if __name__ == "__main__":
    test_login_success()

```

backend unit teszt kód:

```
24 public void Init()
25 {
26     var options = new DbContextOptionsBuilder<TesztContext>()
27         .UseInMemoryDatabase(databaseName: "TestDb_" + System.Guid.NewGuid())
28         .Options;
29
30     _context = new TesztContext(options);
31
32     string secretKey = "EZ_EGY_NAGYON_HOSSZU_BIZTONSAGOS_MULCS_AMI_MINIMUM_32_KARAKTER!!!";
33
34     var settings = new Dictionary<string, string>
35     {
36         {"Jwt:SecretKey", secretKey},
37         {"Jwt:Issuer", "TestIssuer"},
38         {"Jwt:Audience", "TestAudience"},
39         {"EmailSettings:AppBaseUrl", "https://localhost:7133"}
40     };
41
42     _config = new ConfigurationBuilder()
43         .AddInMemoryCollection(settings)
44         .Build();
45
46     _jwtSettings = new JwtSettings
47     {
48         SecretKey = secretKey,
49         Issuer = "TestIssuer",
50         Audience = "TestAudience"
51     };
52
53     _controller = new RegistryController(_context, _jwtSettings, _config);
54 }
55
56 [TestMethod]
57 public async Task Login_ValidUser_ReturnsOk()
58 {
59     var user = new User
60     {
61         Username = "tesztuser",
62         Email = "teszt@example.com",
63         Privilege = 0,
64         PasswordHash = PasswordHasher.HashPassword("123456")
65     };
66
67     _context.Users.Add(user);
68     await _context.SaveChangesAsync();
69
70     var dto = new LoginDTO
71     {
72         Username = "tesztuser",
73         Password = "123456"
74     };
75
76     var result = await _controller.Login(dto);
77     var okResult = result as OkObjectResult;
78
79     Assert.IsNotNull(okResult);
80     Assert.AreEqual(200, okResult.StatusCode);
81 }
82
83 [TestMethod]
84 public async Task Login_UnverifiedEmail_ReturnsUnauthorized()
85 {
86     var user = new User
87     {
88         Username = "unverified",
89         Email = "unverified@example.com",
90         Privilege = 1,
91         PasswordHash = PasswordHasher.HashPassword("123456")
92     };
93
94     _context.Users.Add(user);
95     await _context.SaveChangesAsync();
96
97     var dto = new LoginDTO
98     {
99         Username = "unverified",
100         Password = "123456"
101     };
102
103     var result = await _controller.Login(dto);
104     var unauthorized = result as UnauthorizedObjectResult;
105
106     Assert.IsNotNull(unauthorized);
107     Assert.AreEqual(401, unauthorized.StatusCode);
108 }
109
110 [TestMethod]
111 public async Task Register_ValidUser_CreatesUserWithToken()
112 {
113     var dto = new RegistryDTO
114     {
115         Username = "ujuser",
```

```

116         Email = "ujuser@example.com",
117         Password = "123456"
118     };
119
120     var result = await _controller.Register(dto);
121
122     var savedUser = _context.Users.FirstOrDefault(u => u.Username == "ujuser");
123
124     Assert.IsNotNull(savedUser);
125     Assert.AreEqual("ujuser@example.com", savedUser.Email);
126     Assert.AreEqual(1, savedUser.Privilege);
127     Assert.IsFalse(string.IsNullOrEmpty(savedUser.EmailVerificationToken));
128     Assert.IsNotNull(savedUser.VerificationTokenExpiry);
129     Assert.IsTrue(savedUser.VerificationTokenExpiry > DateTime.UtcNow);
130 }
131
132 [TestMethod]
133 public async Task Register_DuplicateUsername_ReturnsBadRequest()
134 {
135     _context.Users.Add(new User
136     {
137         Username = "tesztuser",
138         Email = "regi@example.com",
139         PasswordHash = PasswordHasher.HashPassword("123456"),
140         Privilege = 1
141     });
142     await _context.SaveChangesAsync();
143
144     var dto = new RegistryDTO
145     {
146         Username = "tesztuser",
147         Email = "uj@example.com",
148         Password = "123456"
149     };
150
151     var result = await _controller.Register(dto);
152     var badRequest = result as BadRequestObjectResult;
153
154     Assert.IsNotNull(badRequest);
155     Assert.AreEqual(400, badRequest.StatusCode);
156 }
157
158 [TestMethod]
159 public async Task Register_DuplicateEmail_ReturnsBadRequest()
160 {
161     _context.Users.Add(new User

```

```

162     {
163         Username = "regiuser",
164         Email = "teszt@example.com",
165         PasswordHash = PasswordHasher.HashPassword("123456"),
166         Privilege = 1
167     });
168     await _context.SaveChangesAsync();
169
170     var dto = new RegistryDTO
171     {
172         Username = "ujuser",
173         Email = "teszt@example.com",
174         Password = "123456"
175     };
176
177     var result = await _controller.Register(dto);
178     var badRequest = result as BadRequestObjectResult;
179
180     Assert.IsNotNull(badRequest);
181     Assert.AreEqual(400, badRequest.StatusCode);
182 }
183 }
184

```